

構造持続における M （有効維持余力）の操作的定式化

— 構造持続の収支原理のための operational resource mapping —

Akihito Sunagawa

2026年4月12日

要旨

本補論は、構造持続の最小形式 $S = Me^{-L}$ のうち、支える側の項 M を有効維持余力として整理する。ここで重要なのは、 M の存在そのものが新しいということではない。構造が維持できない原因を資源不足や余力不足として見る考え方は、従来から自然に用いられてきた。本理論の新規性は、支える側の M だけでは説明できなかった維持可能領域の縮小を、 L や B として分離した点にある。本補論の役割は、その既知のリソース側を、主理論の座標と矛盾しない形で記録する操作的整理である。

具体的には、内部の維持能力成分を $M^{\text{int}} = (M_{\text{buffer}}^{\text{int}}, M_{\text{recovery}}^{\text{int}}, M_{\text{reconfiguration}}^{\text{int}})$ 、外部供給 channel を $M^{\text{external}} = (M_{\text{ext} \rightarrow \text{buffer}}, M_{\text{ext} \rightarrow \text{recovery}}, M_{\text{ext} \rightarrow \text{reconfiguration}})$ と分け、raw resource R から維持能力成分への写像 γ_i 、内部能力と外部供給を合わせる集約 $\widetilde{M}_j = A_j(M_j^{\text{int}}, M_{\text{ext} \rightarrow j})$ 、複数の effective component を結合する Φ を通じて、系の構造持続量を

$$S = \Phi(\widetilde{M}_{\text{buffer}}, \widetilde{M}_{\text{recovery}}, \widetilde{M}_{\text{reconfiguration}}) e^{-L}$$

と書き直す。

本補論の役割は、普遍法則そのものを主張することではなく、構造消耗量と回復量の収支を現実ドメインへ写すときの M 側の操作化を与えることである。ここでの M は、その時点で、その対象構造の維持に実際に使える有効量である。成分分解は、この有効量を応用上読みやすくするための補助的な診断道具であり、必須の理論核ではない。

本補論は新しい普遍法則の証明ではない。また、経験的 pilot を完了した論文でもない。本補論の位置づけは、構造持続の収支原理の回復量・資源入力を実ドメインで測るための support-side operational mapping である。

1 はじめに

構造持続理論の既存の分冊は、主として L の側、すなわち「構造を維持できる状態の集合がどれだけ削られたか」を扱ってきた。Paper 1 は、構造消耗尺度の公理系 B1-B4 から対数比の一意性定理を導き、残存可能性の指数表現

$$m(V^{(n)}) = m(V^{(0)})e^{-Ln}$$

を恒等式として与えた。条件つき導出補論は、この指数表現が A1-A2 のもとで恒等式として成り立ち、段階構造消耗生成過程の弱依存のもとでも指数境界として安定に保たれることを述べた。LLM companion I は、LLM の長期対話において未整理矛盾が有効推論経路を削る過程を 810 試行と対話実験で示し、LLM companion II は、LoRA ベース継続学習において、前提更新が依存知識の再編を壊す破滅的忘却を示した。

これらはいずれも L という「削られる側」の座標を具体化する方向に集中している。

本補論は新しい普遍法則の証明ではない。本補論は、支える側の操作的座標系である。構造持続の最小形式と条件つき導出補論が与えた structural consumption L 、および LLM companion I/II で経験的に観察された構造消耗と支援の相互作用を前提として、構造消耗を回復する資源・修復入力を実ドメインでどう記録するかを問う。

Paper 1 の最小形式 $S = Me^{-L}$ には、従来から理解されてきたリソース側が残る。それが有効維持余力 M である。 M は「 L とは別側で、構造がどれだけ持ちこたえられるか」を担う支え側の量として導入されたが、既存分冊ではその操作的な測り方はほとんど議論されていない。補論「構造持続写像の標準手順」は運用展開

$$S = N_{\text{eff}}^{(0)} \times (\mu/\mu_c) \times e^{-L}$$

を与え、 M を初期有効選択肢多様性 $N_{\text{eff}}^{(0)}$ と有効余力 μ の積として解体しているが、この展開のうち、何が「耐える」作用で、何が「戻す」作用で、何が「作り変える」作用で、何が「外から支える」作用かは、区別されないままである。

1.1 スカラー M の限界

現行のスカラー M だけでは、次のような観察を十分に記録できない。

- ・ 資源が潤沢にあるにもかかわらず、機能が維持できない。

- raw resource R は同じでも、その時点で対象構造に投入できる有効量が異なる。
- 崩壊の仕方が系ごとに質的に異なる (即時、徐々、相転移)。
- 余力が、耐える、戻す、作り変える、外から支える、などの異なる維持機能として現れる。

これらはいずれも「削られ方」の差ではなく、「支え方」の差である。L 側の解像度をいくら上げても、この差は出てこない。現行スカラー M は、これらの質的差を単一数量に押し込めてしまい、区別の余地を残していない。

1.2 本補論の問い

本補論の問いは、一文に要約される。

raw resource R のうち、どれだけが対象構造を維持するための有効量 M として、その時点で使えるのか。

この問いに答えるには、 M を raw resource と同一視せず、「構造が持ちこたえるために実際に使える有効量」として読む必要がある。成分分解はそのための一つの操作的手段であり、本補論はその最小の定式化を与えることを目的とする。

1.3 立場と範囲

本補論の立場は、主理論 spine、LLM companion、および補論群と整合するように、以下のように限定する。

- 本補論は M の完全理論ではない。現行スカラー M を raw resource から区別し、必要に応じて維持能力成分ベクトルへ分解するための分析フレームである。
- 本補論は構造持続の収支原理の中核ではなく、その回復量・資源入力を実ドメインへ写す操作化層である。
- 本補論では、 M の操作的 readout が baseline にない情報を持つかを検査標的に置く。崩壊プロファイルや時間発展主張は後続の拡張とみなす。
- 本補論は最初のドメインとしてソフトウェア / SaaS を置き、four-domain comparison や普遍理論の宣言には進まない。
- 本補論は Paper 1 §3 の対数比の一意性定理と同じ設計原理 (Cauchy 関数方程式と連続性から一意関数形を強制する) を M 側に移植する候補を持つ。§2.5 では、この表現補題候補を本補論の短い theoretical pointer として置き、証明と公理列挙の詳細は別稿の補論に委ねる。

- 本補論は仕様固定構造層の普遍法則宣言を行わない。ソフトウェアは構造推定層として扱う。

本補論で扱う対象構造は、Paper 1 §2 の適用可能性条件 P1–P5 を満たすもの、すなわち観測前に対象構造・測度・制約列・時間地平が事前固定された構造維持問題に限る。観測後に F、 Σ 、R、成分集合を選び直してよいなら、本補論の予測は事後的適合によって空虚化するからである。

1.4 LLM companion I / II との概観的接続

§2 の維持能力成分の分解を置くと、LLM companion I と II の具体的観察は、それぞれ M の異なる成分として自然に読み直される。詳細な対応は §3 に委ねるが、概観を述べておく。

- LLM companion I の scope-as-repair および attribution-as-repair は、「source 分離」という最小の整理作用として働き、未整理矛盾による有効 L 蓄積を局所的に削減する。本補論の枠組みでは、これは base LLM の内部 M_{recovery} 能力を prompt design が誘導した結果、すなわち in-context M_{recovery} として読める。
- LLM companion I の外部代謝 ON/OFF 実験は、対話 LLM 単体では欠けていた M_{recovery} が外部プロセスから供給された効果の直接観察として位置づけられる。本補論の記法では $M_{\text{ext} \rightarrow \text{recovery}}$ である。
- LLM companion II の LoRA 逐次更新が「蓄積ではなく上書き」に振る舞う結果は、パラメータ更新が partial $M_{\text{reconfiguration}}$ に近い作用を持つが M_{recovery} を代替しないという、成分分離の経験的支持として読める。
- LLM companion II の F-v2c は依存構造に沿った選択的再提示によって外部 M_{recovery} を運用した結果、F-multi は空間分離による部分的 $M_{\text{buffer}} / M_{\text{reconfiguration}}$ の模倣と読める。

これらの接続は、本補論の維持能力成分の分解が既存観察に対する事後的再記述にとどまらず、異なる分冊で観察された現象を共通の座標で読むための座標系を提供することを示唆する。詳細な対応表と、各成分の LLM companion I/II における具体的指標は §3 で与える。

2 最小形式

2.1 F / Sigma / R / M の 4 層

構造を維持できるかどうかは、単に raw resource がどれだけあるかの問題ではない。何を守ろうとしているのか、どの構造を通して守るのか、その資源がその時点でどれだけ有効維持余力に変換されているのかによって決まる。本補論では、この区別を次の 4 層で明示する。

- F : 守りたい機能 (*target function*)
- Σ : その機能を担う構造
- R : 資源素材 (*raw resource stock*)
- M : 有効維持余力 (*effective maintenance amount / capacity*)

ここで F と Σ は、Paper 1 の $V^{(0)}$ および測度 m の具体化に対応する。どの構造の持続を問題にしているかを決めるのが F と Σ である。 R は、系が素材として持っている stock である。予算、人員、時間、エネルギー、容量、冗長要素などが該当する。 M は、その R のうち「 F を守るために、その時点で実際に使える有効量」である。

重要なのは、 R と M を混同しないことである。現金はあるが承認権限が詰まっている、病床はあるが看護師がいない、キャッシュはあるが設定ミスで効かない、という状況はすべて「 R はあるが M は小さい」と書ける。本補論の維持能力成分の分解が意味を持つのは、この区別が最初から明示されているからである。

2.2 維持能力成分と供給 channel の定義

M の最小意味は、対象構造の維持に使える有効量である。応用上、この有効量をどの機能として使えるかを区別したい場合には、内部の維持能力成分と外部供給 channel に分けて扱う。集約関数 Φ が直接受け取る同列の座標は $M_{\text{buffer}}, M_{\text{recovery}}, M_{\text{reconfiguration}}$ の三つである。外部供給は第四の維持能力成分ではなく、外部から他成分を供給する channel / externalization profile として扱う。以下では、記号を短くするため external を ext と略記する。

定義 2.1 (維持能力成分と外部供給 channel).

$$M^{\text{int}} = (M_{\text{buffer}}^{\text{int}}, M_{\text{recovery}}^{\text{int}}, M_{\text{reconfiguration}}^{\text{int}}),$$

$$M^{\text{external}} = (M_{\text{ext} \rightarrow \text{buffer}}, M_{\text{ext} \rightarrow \text{recovery}}, M_{\text{ext} \rightarrow \text{reconfiguration}}).$$

各成分の意味は次の通り。

記号	英名	維持能力成分
$M_{\text{buffer}}^{\text{int}}$	internal buffer capacity	base system 自身が今この瞬間に持ちこたえる力 (耐える)
$M_{\text{recovery}}^{\text{int}}$	internal recovery capacity	base system 自身が壊れた部分を元の構造に戻す力 (戻す)
$M_{\text{reconfiguration}}^{\text{int}}$	internal reconfiguration capacity	base system 自身が同じ F を保ったまま構造を再編する力 (作り変える)
$M_{\text{ext} \rightarrow \text{buffer}}$	externally supplied buffer capacity	外部 channel が buffering を供給する量
$M_{\text{ext} \rightarrow \text{recovery}}$	externally supplied recovery	外部 channel が recovery / repair を供給する量
$M_{\text{ext} \rightarrow \text{reconfiguration}}$	externally supplied reconfiguration capacity	外部 channel が reconfiguration を供給する量

ここで強調すべきなのは、buffer, recovery, reconfiguration は「何を供給するか」を表す維持能力成分であり、external は「誰が供給するか」を表す channel であるという点である。予算や人員や時間そのものは source であって、 M_{buffer} などの成分ではない。予算が増えたからといって、それが自動的に buffer capacity や recovery capacity に変換されるわけではない。どの成分に、どれだけ、どのように変換されるかを操作的に記述するのが、次節で導入する写像 γ_i の役割である。

以降では、内部能力と外部供給を合わせた effective component profile を

$$\widetilde{M}_j = A_j(M_j^{\text{int}}, M_{\text{ext} \rightarrow j}),$$

$$j \in \{\text{buffer, recovery, reconfiguration}\}$$

と書く。 A_j は少なくとも非負かつ各引数に単調非減少な集約である。最も単純には加法型 $A_j(u, v) = u + v$ と置けるが、本補論の検査標的はこの特定形に依存しない。

2.3 $M_{\text{reconfiguration}}$ に関する重要な制限

$M_{\text{reconfiguration}}$ に関しては、本補論では以下の強い制限を置く。

$M_{\text{reconfiguration}}$ は、target function F を保つ範囲の再編に限定される。 Σ が別の Σ' に置換されることを扱うが、置換後も F は保持されていなければならない。 F 自体が変わる遷移は $M_{\text{reconfiguration}}$ の対象外であり、別稿の課題として残す。

この制限が必要なのは、再編を無制限に許すと理論が空虚化するためである。たとえば、ある企業が破綻後に別業種で存続した場合、それは元の target function を保ったのではなく、別の F に乗り換えたのである。これを「再編して生き延びた」と扱うと、どんな結果でも事後的に説明できてしまう。

この制限は、Paper 1 §2 が「基体そのものの消滅ではなく、ある構造としての持続の失敗」を扱うと述べた立場と一貫する。 F 自体の遷移は、本補論の枠組みではなく、構造持続の集合値力学的表現 (別補論) の R_t 作用のうち target structure 自体を書き換える部分として、将来の拡張対象となる。

2.4 Raw resource から維持能力成分への写像 γ_i

R から各 M_i への変換を、写像 γ_i で与える。

定義 2.2 (維持能力成分への変換写像).

$$M_j^{\text{int}} = \gamma_j^{\text{int}}(R, \Sigma, F),$$

$$M_{\text{ext} \rightarrow j} = \gamma_{\text{ext} \rightarrow j}(R, \Sigma, F),$$

$$j \in \{\text{buffer, recovery, reconfiguration}\}.$$

各 γ は次の条件を満たすものとする。

- 非負性: $\gamma(R, \Sigma, F) \geq 0$
- 弱単調性: R の関連成分について単調非減少
- 相対性: F および Σ を固定したときに定まる
- 事前固定性: 経験的应用では、結果観測前に定義される

ここで「関連成分」とは、 R のうちその維持能力成分または外部供給 channel に寄与しうる成分を指す。たとえばソフトウェアドメインでは、冗長サーバは $M_{\text{buffer}}^{\text{int}}$ に寄与しうるが、rollback 導線の整備状況なしには $M_{\text{recovery}}^{\text{int}}$ には寄与しない。vendor contract は $M_{\text{ext} \rightarrow \text{recovery}}$ に寄与しうるが、incident response と接続されていなければ実効化しない。どの R 成分がどの維持能力成分に寄与しうるかは domain 固有の設計問題であり、本補論はドメイン横断の普遍的対応表を主張しない。

γ_i を導入することで、本補論は次の観察を理論内で書けるようになる。

$$R \text{ は大きいが、} \gamma_j^{\text{int}}(R, \Sigma, F) \approx 0$$

または

$$\gamma_{\text{ext} \rightarrow j}(R, \Sigma, F) \approx 0.$$

すなわち、「資源は潤沢にあるが、機能維持能力にはなっていない」という事態が、 M_i の小ささとして定量的に書ける。この形は、§1.1 で挙げた「資源があるのに機能しない」という観察に対する形式的な位置づけを与える。

2.5 有効維持余力と集約関数 Φ

系の有効維持余力 M_{eff} を、維持能力成分ベクトルの集約量として定める。

定義 2.3 (有効維持余力).

$$M_{\text{eff}} = \Phi(\widetilde{M}_{\text{buffer}}, \widetilde{M}_{\text{recovery}}, \widetilde{M}_{\text{reconfiguration}}).$$

Φ は、少なくとも非負性と各 effective component に関する単調非減少性を満たす集約関数である。

本補論では Φ の関数形を一意に固定しない。むしろ、表現規律として、いくつかの候補族を明示し、その選択に対して M-profile の読みが頑健かを §6 で検査する。

候補の一つは積型である。scale separability (各成分を独立にスケールしたときの影響が他成分を変えずに乗法的に分離できる性質) と multiplicative composition ($h_i(\lambda\mu) = h_i(\lambda)h_i(\mu)$) を追加仮定として置くと、連続性と正規化から Φ は積型

$$\Phi(q) = \prod_i q_i^{\alpha_i}$$

に制限される。一次同次性 $\Phi(cq) = c\Phi(q)$ を追加すれば $\sum_i \alpha_i = 1$ が従う。ただし、この仮定は強い。scale separability を置いた時点で、積型へかなり寄っている

る。したがって、これは Paper 1 §3 と同じ強度の非自明な一意性定理ではなく、積型を選ぶ場合の表現補題候補として扱う。

他の候補として、CES family

$$\Phi_{\rho}(q) = \left(\sum_i \alpha_i q_i^{\rho} \right)^{1/\rho}$$

を置ける。 $\rho = 1$ は加法型、 $\rho \rightarrow 0$ は積型、 $\rho \rightarrow -\infty$ は bottleneck / Leontief 型に対応する。各成分が代替不能な regime では bottleneck 型

$$\Phi(q) = \min_i w_i q_i$$

が自然である。

この関数族の整理は、Paper 1 §3 の対数比の一意性定理と方法論上は対応するが、強度は異なる。Paper 1 では構造消耗尺度の加法性が対数形を強く拘束した。本補論では、support 側の結合様式そのものが経験的・設計的選択を含むため、積型・CES・bottleneck のどれを採るかを理論だけで一意化しない。対応は次の通り。

項目	Paper 1 §3	本補論 §2.5
対象	残存比率 $r \in (0, 1]$ 上の構造消耗 f	正規化能力 $q_i \in \mathbb{R}_{>0}$ 上の集約 Φ
関数方程式	additive Cauchy	multiplicative Cauchy は積型候補の十分条件
一意な関数形	$f(r) = -k \ln r$	積型・CES・bottleneck の候補族
残る経験量	各ドメインにおける $m(V)$ の具体化	各ドメインにおける α_i, w_i, ρ とその頑健性

この骨格対応により、本補論の維持能力成分の分解は単なる分類表ではなく、Paper 1 の方法論を support 側に移植するための候補族を持つ。ただし、本補論の load-bearing claim は積型の証明ではない。以降の節では Φ を単調非減少な集約として扱い、積型・CES・bottleneck 型のどれを採るかは domain 固有の経験的推定と §6 の robustness validation に委ねる。M-profile support は、この選択に対して robust であるべきである。

以上により、本補論の最小形式は次で与えられる。

定義 2.4 (本補論の最小形式).

$$\begin{aligned} S &= M_{\text{eff}} e^{-L} \\ &= \Phi(\widetilde{M}_{\text{buffer}}, \widetilde{M}_{\text{recovery}}, \widetilde{M}_{\text{reconfiguration}}) e^{-L}. \end{aligned}$$

この式は Paper 1 の $S = M e^{-L}$ を否定するものではない。スカラー M を M_{eff} で置き換え、 M_{eff} を effective component profile の集約として再構成しただけであり、スカラー M は本式の低粒度の要約として回収される。

2.6 Lean で閉じている範囲

本補論の維持能力成分の分解は、`Survival/MaintenanceComponentDecomposition.lean` で形式化されている。Lean 側で閉じているのは、経験的な proxy の妥当性ではなく、M 側の表現文法と、M interface 上の表現定理である。

Lean では、維持能力成分を

$$\{\text{buffer, recovery, reconfiguration}\}$$

の三値型として定義し、供給 channel を

$$\{\text{internal, external}\}$$

の二値型として別に定義する。これにより、external は第四の維持能力成分ではなく、三つの成分を供給する channel であることが型レベルで固定される。

さらに Lean では、任意の raw mechanism を直接分類するのではなく、それが M 側の観測 interface に入ったときの効果を component profile として扱う。したがって「第四成分が存在しない」という主張は、現実世界の原因機構が三種類しかないという主張ではない。正確には、M として観測される効果は $M_{\text{buffer}}, M_{\text{recovery}}, M_{\text{reconfiguration}}$ の三座標に還元され、同じ三座標を持つ二つの mechanism を M interface 内部で区別する独立な第四座標は存在しない、という表現定理である。三座標で表現できない候補が現れた場合、それは第四の M 成分ではなく、対象構造・測度・環境・同一性条件など、M interface の外側に置くべき候補として扱う。

この主張はさらに、標準商としても定式化される。Lean 側では、raw mechanism を「同じ三成分 profile を持つなら同一視する」同値関係で割った `ObservationQuotient` を作り、任意の M-side readout がこの標準商を一意に通ることを証明している。したがって三成分 profile は単なる便利な座標ではなく、M interface 上で識別可能な差分を失わない最小完全観測商として機能する。

対応する定理は次の通りである。

Lean entry point	内容
<code>MaintenanceComponent.exhaustive</code>	維持能力成分は <code>buffer / recovery / reconfiguration</code> の三つで尽きる
<code>SupplyChannel.exhaustive</code>	供給 <code>channel</code> は <code>internal / external</code> の二つで尽きる
<code>componentProfile_ext</code>	<code>component profile</code> は三成分の値で一意に決まる
<code>supplyProfile_ext</code>	<code>supply profile</code> は <code>internal/external × 三成分</code> の六座標で一意に決まる
<code>fromInternalExternal_internalProfile</code>	任意の <code>external profile</code> は <code>internal profile</code> と <code>external profile</code> に分解できる
<code>fromInternalExternal_eq_iff_effectiveProfile</code>	その <code>internal/external</code> 分解は一意である <code>internal/external</code> の供給を成分ごとに集約し、 <code>effective component profile</code> を作る
<code>effectiveMaintenance_nonneg</code>	非負供給と非負性保存 aggregator のもとで、 <code>effective maintenance capacity</code> は非負になる
<code>MaintenanceInterface.everyMechanismIsThreeComponentRepresentation</code>	<code>MaintenanceInterface</code> に入力される <code>ComponentRepresentation</code> は三成分 <code>profile</code> として表現される
<code>MaintenanceInterface.observationalEquivalenceThreeCoordinates</code>	<code>MaintenanceInterface</code> の観測同値性は三成分の一致と同値である
<code>MaintenanceInterface.noFourthObservableCoordinate</code>	三成分で一致しない <code>M</code> の <code>mechanism</code> を分ける第四 <code>M</code> 座標は <code>interface</code> 内部には存在しない
<code>MaintenanceInterface.maintenanceReadoutIsNotSideOfSameThreeCoordinates</code>	任意の <code>M-side of same three coordinates</code> する <code>mechanism</code> を区別できない
<code>MaintenanceInterface.factors_through_observationQuotient_M_respects_equality</code>	観測量が標準商を通る <code>Quotient_M</code> 観測同値性を尊重することは同値である
<code>MaintenanceInterface.maintenanceReadout_factors_through_observationQuotient</code>	任意の <code>M-factors through</code> 標準商 <code>ObservationQuotient</code> を通る
<code>MaintenanceInterface.quotientFactorization</code>	標準商を通る <code>factorization</code> は一意である
<code>MaintenanceInterface.quotientProfile_ext</code>	標準商の <code>effective</code> は三成分 <code>profile</code> によって忠実に表現される

Lean entry point	内容
<code>MaintenanceInterface.quotientProfile</code>	標準商の等号は complete profile の等号と同値である
<code>MaintenanceInterface.outsideInterface</code>	同列成分 ϕ を持つ $\text{these_observationalEq}$ を区別する追加量は M interface を factor しない
<code>MaintenanceInterface.outsideInterface</code>	成分が一致しても distinguish を区別する追加量は M interface の外側に属する
<code>PartialMaintenanceInterface.representable</code>	候補 table に入っている profile に表現されるか、M interface の外側に置かれる

この Lean 化が閉じるのは、「本補論の記法体系では、M 側の同列成分は三つであり、external はそれらを生供給する channel である」という構文的・代数的事実に加えて、「M interface 上で観測される効果は三成分 profile によって完全に決まる」という表現定理である。したがって、次は Lean の範囲外に残す。

- 各ドメインでこの三成分が自然に測れること。
- γ_i 、 A_j 、 Φ の経験的に最良な形。
- software / SaaS における component signal の妥当性。

つまり、Lean は本補論の「外部供給を第四成分にしない」という文法と、「M interface 内部では三成分以外の独立座標を持たない」という表現定理を閉じる。さらに、M interface 上の任意の valid readout は三成分 profile から作られる標準商を一意に通る。経験的価値は、§6 の事前固定 validation で判定する。

2.7 補論 B との関係

本補論の枠組みが補論 B の運用展開

$$S = N_{\text{eff}}^{(0)} \times (\mu/\mu_c) \times e^{-L}$$

と重複しないように、両者の関係を明示しておく。

補論 B の量	本補論側の最も近い位置
μ/μ_c	M_{buffer} (buffering margin) に最も近い

補論 B の量	本補論側の最も近い位置
$N_{\text{eff}}^{(0)}$	初期選択枝多様性。 $M_{\text{reconfiguration}}$ の上流に位置するが、 $M_{\text{reconfiguration}}$ と同一ではない
(補論 B では未分離)	M_{recovery} は補論 B の μ 側に畳み込まれていた作用として切り出される
(補論 B では未分離)	$M_{\text{reconfiguration}}$ は $N_{\text{eff}}^{(0)}$ と μ のあいだに埋まっていた再編作用として切り出される
(補論 B には対応物なし)	external channel は開放系の外部供給として補論 B の外に置かれる

重要な注意として、静的な $N_{\text{eff}}^{(0)}$ をそのまま $M_{\text{reconfiguration}}$ に吸収しないこと。ドメインが再編を通じて選択枝を再生させるケース以外では、 $N_{\text{eff}}^{(0)}$ と $M_{\text{reconfiguration}}$ は別物として保つ方が安全である。本補論は補論 B を上書きするのではなく、補論 B の右辺の M 側を維持能力成分へ分解して再解釈するものとして位置づける。

3 LLM companion I / II の成分対応

§2 では、有効維持余力を、内部の維持能力成分

$$M^{\text{int}} = (M_{\text{buffer}}^{\text{int}}, M_{\text{recovery}}^{\text{int}}, M_{\text{reconfiguration}}^{\text{int}})$$

と外部供給 channel

$$M^{\text{external}} = (M_{\text{ext} \rightarrow \text{buffer}}, M_{\text{ext} \rightarrow \text{recovery}}, M_{\text{ext} \rightarrow \text{reconfiguration}})$$

に分けた。本節では、この component / channel 分解を LLM companion I と II の観察に対応づける。ただし、本節の対応は成分値の直接推定ではない。各実験で観察された差分を、どの成分の不足または外部供給を示す indicator として読むのが安全である。

3.1 対応の原則: 成分と担い手を分ける

本節で最も重要なのは、維持能力成分と担い手を混同しないことである。 $M_{\text{buffer}}, M_{\text{recovery}}, M_{\text{reconfiguration}}$ は「どの維持能力成分が働くか」を表す。これに対して、その成分を担うのが base system 自身なのか、prompt 内の表現なの

か、外部プロセスなのかは別問題である。§2 の formal notation では、外部供給分を $M_{\text{ext} \rightarrow \text{buffer}}$, $M_{\text{ext} \rightarrow \text{recovery}}$, $M_{\text{ext} \rightarrow \text{reconfiguration}}$ と書く。

したがって、外部システムが repair 型の作用を供給する場合、本補論ではこれを

$$M_{\text{ext} \rightarrow \text{recovery}}$$

と書く。これは「外部 channel が recovery 成分を供給している」ことを表す記法である。ext は外部支援の channel を表し、 M_{recovery} は供給される維持能力成分を表す。つまり、

- in-context M_{recovery} : base system 内の γ_{recovery} を prompt design によって誘導する。
- $M_{\text{ext} \rightarrow \text{recovery}}$: 外部プロセスが repair / resolution を担い、その結果を base system に供給する。

この区別により、LLM companion I の scope-as-repair と外部代謝 ON/OFF を同じ「repair 的效果」として見つつ、供給階層の違いを失わずに記述できる。

3.2 LLM companion I: 推論時矛盾と外部代謝

LLM companion I は、LLM 推論における未整理矛盾の効果を扱った。本補論の観点から見ると、LLM companion I は主に次の二つを示している。

第一に、未整理矛盾は L 側の構造消耗として働く。第二に、その構造消耗を抑えるには、矛盾を範囲づける repair 型の作用が必要である。この repair は、prompt 内で誘導される場合もあれば、外部代謝プロセスによって供給される場合もある。

3.2.1 Scope-as-repair / attribution-as-repair: in-context M_{recovery}

Exp.40 は、矛盾の有無ではなく、矛盾が task 外に範囲づけられているかどうかを前向きに比較した。32K 文脈に固定し、zero_sanity, scoped, subtle, structural を各 50 試行で比較した結果は次である。

条件	正答率
zero_sanity	50/50 = 1.00
scoped	50/50 = 1.00
subtle	23/50 = 0.46
structural	0/50 = 0.00

scoped が zero_sanity と同水準に戻り、subtle と structural が崩れるという結果は、単に矛盾らしき文があるかどうかではなく、その衝突が task から範囲づけられているかどうかことが重要であることを示した。本補論の語彙では、これは base LLM の in-context γ_{recovery} が prompt design によって誘導された indicator と読める。

Exp.42 は、この repair をさらに分解した。strong_scope, medium_scope, weak_scope, subtle の四段階で、正答率は次のようになった。

条件	正答率	exact wrong-sum adoption
strong_scope	50/50 = 1.00	0/50
medium_scope	49/50 = 0.98	0/50
weak_scope	42/50 = 0.84	1/50
subtle	10/50 = 0.20	25/50

row-level では、exact wrong-sum adoption が subtle の $25/40 \text{ mistakes} = 0.625$ から、weak_scope の $1/8 \text{ mistakes} = 0.125$ 、medium_scope / strong_scope の 0 へ落ちた。これは、明示命令だけでなく、参照元 attribution という最小の source label が contradiction-taking を大きく抑えることを示す。

この効果は外部プロセスによるものではない。prompt 内の source / dataset / temporal marker が、base LLM の解釈過程を修復方向へ誘導している。したがって、本補論ではこれを in-context M_{recovery} の indicator と呼ぶ。

Exp.41 は、この方向が gpt-4.1-mini 固有でないことを検査した。gpt-4.1-nano では scoped=27/30 = 0.90, structural=1/30 = 0.03、gemini-3.1-flash-lite-preview では scoped=30/30 = 1.00, structural=14/30 = 0.47 であり、二つの primary model の両方で scoped > structural が成立した。ただし subtle と structural の大小関係はモデル依存であった。したがって、本補論が受け取るべき invariant は、固定された subtle/structural の大小関係ではなく、scope marker が repair 的に働くという狭い方向である。

特に gpt-4.1-nano では subtle=30/30 = 1.00 と天井に張りついたため、secondary ordering は固定的な invariant として扱わない。

3.2.2 外部代謝 ON/OFF: $M_{\text{ext} \rightarrow \text{recovery}}$

LLM companion I の対話実験では、未整理矛盾を外部で検出し、時間ラベル付きの更新対として整理する代謝パイプラインを ON/OFF で比較した。ここで ON は、矛盾更新を外部プロセスが整理し検索可能な形に保持する条件であり、OFF は同じ矛盾を未整理のまま混在させる条件である。

gemma3:27b の 180 ターン実験では、対話 LLM と代謝 LLM は同一モデルであるが、代謝は対話呼び出しとは別のプロセスとして行われる。規則＋事実の合算は次であった。

条件	規則＋事実	合算
ON		73.3%
NC		56.7%
OFF		21.1%

ON vs OFF は $p = 0.0004$, Cohen's $d = 8.80$ であった。qwen3.5:27b の追試では、ON の代謝 LLM に Claude Sonnet を使用し、全体正答率は ON 64.4%、OFF 44.4% であった。

この効果は in-context marker とは階層が異なる。代謝 pipeline が、古い情報と新しい情報の衝突を検出し、旧値 $\rightarrow$ 新値という範囲づけられた形へ変換してから base system に供給している。本補論の語彙では、これは $M_{\text{ext} \rightarrow \text{recovery}}$ の indicator である。

ここで ext は「外部に助けられている」という channel を表し、 M_{recovery} は「供給されている作用が repair / resolution 型である」ことを表す。同じ外部支援でも、冗長サーバを供給するなら $M_{\text{ext} \rightarrow \text{buffer}}$ 、依存再編を供給するなら $M_{\text{ext} \rightarrow \text{recovery}}$ と読むべきである。

さらに、qwen3.5:9b の代謝あり 100 ターン実験では、規則適用と矛盾検出が 87-100% の範囲で振動し、検索成功率は 96% で安定していた。これは単独では強い検証ではないが、 $M_{\text{ext} \rightarrow \text{recovery}}$ が機能し続ける限り、長期の制約蓄積がただちに単調崩壊へ向かわないことの示唆的 indicator である。

3.2.3 Exp.36 / Exp.39: L の質と量の補助観察

Exp.36 と Exp.39 は、本補論の成分対応そのものではなく、L 側の質的構造が重要であることを示す補助観察として扱うのがよい。

Exp.36 は、3 モデル $\times$ 3 δ 水準 $\times$ 3 文脈長 $\times n = 30$ 、合計 810 試行で、文脈長と矛盾の質を操作した。Exp.39 はその中心的方向を prospective comparison として再検査した。これらの結果は、文脈長や制約数だけでは推論性能劣化を説明できず、構造的矛盾の質が大きく効くことを示す。

本補論にとって、この観察は「成分の直接証拠」ではない。むしろ、 $\gamma_i(R, \Sigma, F)$ の入力として、L 側の構造が raw count ではなく質的に効くことを示す背景である。したがって本節では、Exp.36 / Exp.39 を M の成分値に対応づけず、LLM companion I の L-side anchor として扱う。

3.3 LLM companion II: LoRA 継続学習と依存再編

LLM companion II は、前提更新を伴う LoRA ベース継続学習が、知識を蓄積するのか、それとも上書きするのかを検査した。本補論の観点から見ると、LLM companion II は $M_{\text{reconfiguration}}$ と M_{recovery} の分離を鋭く示している。

3.3.1 LoRA 逐次更新: partial $M_{\text{reconfiguration}}$, weak M_{recovery}

LoRA はパラメータを変えるため、局所的には reconfigurative な作用を持つ。新しい課題や前提更新に反応して表現を変えるという意味で、これは partial $M_{\text{reconfiguration}}$ に近い。しかし LLM companion II の主要結果は、その再構成作用が repair / resolution を代替しないことであった。

主要三条件の最終時点の結果は次である。

条件	T5 依存整合性	T5 更新成功率	T5 時点の T1 保持
E-lite	0.189 ± 0.096	0.400 ± 0.173	0.167 ± 0.289
F-v2c	0.333 ± 0.000	0.583 ± 0.144	0.000 ± 0.000
F-multi	0.367	0.500-0.750	0.500

最初の前提更新後、旧知識保持は全条件で急減した。これは、LoRA 更新が新しい信号に反応して表現を変える一方で、既存の派生知識との整合を自律的に取り直す M_{recovery} を十分に持たないことを示す indicator である。

したがって、本補論では LoRA を「partial $M_{\text{reconfiguration}}$ はあるが weak M_{recovery} 」として読む。ここでの $M_{\text{reconfiguration}}$ は §2.3 の制限に従い、target function F を保つ範囲での再編に限る。LoRA が別タスクへ乗り換えた場合、それは本補論の $M_{\text{reconfiguration}}$ ではなく、対象 F の変更である。

3.3.2 F-v2c: $M_{\text{ext} \rightarrow \text{recovery}}$

F-v2c は、前提と依存属性の関係を DAG として保持し、前提更新時に下流の依存属性だけを選択的に再提示する。これは、base LoRA update が持たない依存再編を、外部 controller が供給する条件である。

平均値では、依存整合性は E-lite の 0.189 から F-v2c の 0.333 へ改善した。一方、T5 時点の T1 保持は 0.000 であり、旧知識保持そのものは回復しなかった。この組み合わせは、F-v2c が「古いものをそのまま保存する」介入ではなく、「現在有効な前提に対して下流知識を整合させ直す」介入であることを示す。

本補論の語彙では、F-v2c は $M_{\text{ext} \rightarrow \text{recovery}}$ の indicator である。外部 controller が依存構造を持ち、base training process に対して repair-like な再提示を供給する。ただし、F-v2c は再提示件数や除外ポリシーも同時に変えているため、依存構造だけ

の寄与を完全に分離した実験ではない。この点は §6 の empirical route で、将来の ablation として扱う。

3.3.3 F-multi: partial M_{buffer} + partial $M_{\text{reconfiguration}}$

F-multi は、現在知識と過去知識を別々のアダプタに分離する条件である。単一アダプタにすべてを混在させるのではなく、保持用の部分空間と現在用の部分空間を分けるため、本補論では partial M_{buffer} と partial $M_{\text{reconfiguration}}$ の合成として読める。

F-multi は T5 時点の T1 保持を 0.500 まで上げ、E-lite や F-v2c にはない非ゼロ保持を与えた。ただし、これは理想振り分け、すなわち対象例が現在知識側か保持知識側かを既知として振り分ける条件である。したがって F-multi の結果は、実運用性能ではなく、空間分離が原理的に保持と更新の衝突を緩和しうることを示す上界 indicator として読むべきである。

F-multi が示すのは、部分空間分離によって一部の M_{buffer} と $M_{\text{reconfiguration}}$ は得られるが、それだけでは高忠実度の長期保持や依存整合の完全な repair には足りない、ということである。

3.4 成分対応表

以上をまとめると、LLM companion I / II の観察は次のように成分 indicator と対応づけられる。

観察	主な indicator	供給階層	主要数値 / 方向	読み方
Exp.40 scoped	in-context M_{recovery}	base LLM + prompt	scoped 50/50, subtle 23/50, structural 0/50	source / dataset scope が contradiction- taking を抑え る
Exp.42 attribution	in-context M_{recovery}	base LLM + prompt	wrong-sum adoption: subtle 25/40 mistakes -> weak 1/8 -> medium/ strong 0	最小 source label が repair を誘導

観察	主な indicator	供給階層	主要数値 / 方向	読み方
Exp.41 width	in-context M_{recovery} の幅 確認	base LLM + prompt	scoped > structural in 2/2 primary models	invariant は scope protection
外部代謝 ON/ OFF	$M_{\text{ext} \rightarrow \text{recovery}}$	out-of-context process	gemma ON 73.3% vs OFF 21.1%; qwen ON 64.4% vs OFF 44.4%	外部整理が未 整理矛盾を時 間ラベルつき に修復
Exp.36 / Exp.39	L-side quality anchor	該当なし	810 試行 + prospective comparison	raw context length ではな く構造の質が 効く
LoRA sequential update	partial $M_{\text{reconfiguration}}$, weak M_{recovery}	parameter update	T1 retention collapse after premise update	再編はあるが 依存 repair は 弱い
F-v2c	$M_{\text{ext} \rightarrow \text{recovery}}$	external controller + training signal	DC 0.189 -> 0.333, T1 retention 0.000	DAG 沿いの選 択的 repair
F-multi	partial M_{buffer} + partial $M_{\text{reconfiguration}}$	adapter separation	T1 retention 0.500 under ideal routing	空間分離によ る保持上界

この表は、既存結果を成分値として再推定するものではない。既存結果は、それぞれの成分が不足している、または外部から供給されている、という方向を示す indicator である。

3.5 LLM companion II §7.5 の三役分離との接続

LLM companion II §7.5 は、持続知能に少なくとも三つの役割が必要であると述べた。第一にパラメータ的再編、第二に外部代謝、第三に応答生成の忠実化である。本補論の維持能力成分の分解は、この三役分離を M 側の言葉で整理し直す。

LLM companion II の役割	本補論の位置づけ	注意
パラメータ的再編	partial $M_{\text{reconfiguration}}$	新しい信号に反応するが、repair を代替しない
外部代謝	$M_{\text{ext} \rightarrow \text{recovery}}$	更新履歴と依存関係を外部で整理する
応答生成の忠実化	output-side realization	M そのものではなく、保持された構造を出力へ反映する段階

ここで output-side realization は第五の維持能力成分ではない。これは、すでに保持・修復・再編された構造が実際の応答へ反映されるかどうかの出力段階であり、本補論の維持能力成分 profile には含めない。

この対応により、LLM companion II の結論は本補論の M-profile 読みに接続する。条件 (i) 内部に長期的な矛盾解消代謝機構を持たず、条件 (ii) 推論呼び出しの境界を越えて信念を持ち越す機構が弱い系では、単なる capacity 増強ではなく、 M_{recovery} の供給が重要になる可能性が高い。

LLM companion I では、これは in-context scope marker または外部代謝として現れた。LLM companion II では、F-v2c の依存 DAG controller として現れた。どちらも、raw resource を増やすのではなく、衝突をどう整理し直すかを変えている。この点で、LLM companion I / II は本補論の基本的読み——raw resource が同じでも、それが対象構造の維持に使える有効量 M へどう変換されるかが異なりうる——への準備的根拠を与える。

3.6 非主張

本節の対応には、次の制限を置く。

1. 本節は $M_{\text{buffer}}^{\text{int}}$, $M_{\text{recovery}}^{\text{int}}$, $M_{\text{reconfiguration}}^{\text{int}}$ や $M_{\text{ext} \rightarrow j}$ の数値を直接推定しない。
2. Exp.40 / 42 / 41 の scope 効果と、外部代謝 ON/OFF の効果が同一機構であるとは主張しない。共通しているのは repair-like な観測帰結であり、供給階層は異なる。
3. gemma3:27b の自己代謝と qwen3.5:27b + Sonnet の外部代謝を同一条件として合算しない。前者は coupled process、後者は teacher-like external process を含む。# F-v2c の改善を依存 DAG の寄与だけに還元しない。再提示件数、除外ポリシー、学習安定性が交絡しうる。

- 4 F-multi を実用性能として読まない。理想振り分け条件で得た上界 indicator である。
- 5 Exp.36 / Exp.39 は M -component の直接証拠ではなく、L-side quality anchor として扱う。
- 6 Software contract-coherence diagnostics、SAT、Mixed-CSP の M -component 解釈は本節では扱わない。これらは §5-6 または別稿で扱う。

4. Software / SaaS における写像

本補論の最初の具体ドメインは、software / SaaS / 継続運用される業務システムである。この選択は、ソフトウェアが最も普遍的な対象であるという主張ではない。むしろ、本補論の目的である M の操作的定式化にとって、software / SaaS が扱いやすい 構造推定層だからである。

理由は二つある。第一に、維持したい機能 F と、それを担う構造 Σ を比較的具体的に書ける。第二に、障害、変更、rollback、MTTR、lead time、deploy history などの観測ログが存在しうる。

4.1 構造推定層としての位置づけ

Software / SaaS は、SAT や Mixed-CSP のような 仕様固定構造層ではない。安全な変更経路や有効運用状態の集合を概念的に置くことはできるが、その残存比率 $m(V^{(n)})/m(V^{(0)})$ を自然測度で直接数えることは難しい。したがって、本補論では software を 構造推定層として扱う。

構造推定層としての勝ち筋は次である。

事前固定した代理構造消耗 $\hat{L}$ と component predictor が、raw size / age / churn / incident count などの基準モデルより、held-out outcome をよく予測するかを見る。

したがって、§4-5 は定理的閉包ではなく、実証可能な写像を定義する節である。ここでの目標は、後続の §6 で validation protocol を置けるだけの $F, \Sigma, R, \hat{L}, M_i, I_i$ を事前に固定することである。

4.2 Target function F

Software / SaaS では、 F を二段階で定義する。

第一に、本補論全体の broad framing として、

$$F_{\text{broad}} = \text{safe change continuity}$$

を置く。これは、可用性、正確性、データ整合性、安全な変更継続、障害からの回復可能性を含む広い機能である。

第二に、最初の empirical pilot の narrow target として、

$$F_{\text{pilot}} = \text{change-introduced bug detection / localization}$$

を置く。これは、変更によって導入される不具合を、release 前後の短い時間窓で検出・局所化できるか、という限定された機能である。

二段階に分ける理由は、broad F が本補論の実務的射程を保つ一方で、narrow F_{pilot} は検証可能性を与えるからである。最初から「安全な変更継続」全体を評価対象にすると、outcome が広すぎて baseline 比較が曖昧になる。まずは bug detection / localization に絞り、そこで構造持続型 proxy が raw baseline を上回るかを検査するのが安全である。

4.3 構造 Σ : code だけではない

Software の Σ は、ソースコードの文字列だけではない。 F を担うのは、コード・設定・データ・運用が組になった関係構造である。

代表的には次が含まれる。

- モジュール境界
- 依存関係
- API contract
- 認証・認可境界
- データ整合性制約
- ストレージ構成
- CI/CD 導線
- 監視と alert 設定
- deploy / rollback path
- incident runbook
- code review checklist
- operational procedure

ここで、LLM companion I における prompt design との対応が明確になる。LLM companion I の in-context scope marker は、LLM 呼び出し内の protocol /

context structure として働いた。Software / SaaS では、runbook、checklist、CI/CD、contract test、review rule、deployment protocol が同じ位置にある。すなわち、これらは単なる raw resource R ではなく、機能を担う構造 Σ の一部である。

したがって、§3 で残した tension への本補論の答えは次である。

$$\text{prompt design / operational protocol / runbook} \in \Sigma.$$

これにより、 $\gamma_i(R, \Sigma, F)$ は context / protocol の違いを自然に受け取れる。たとえば、同じ SRE team time という R があっても、rollback procedure が Σ に存在しなければ M_{recovery} へ変換されにくい。

4.4 Raw resource R

Software / SaaS の R は raw stock であり、それ自体はまだ有効維持余力ではない。

例として次がある。

- server capacity
- cache capacity
- spare compute / storage
- backup storage
- engineer time
- SRE / on-call time
- test infrastructure
- observability tooling
- deployment tooling
- budget
- vendor contract

重要なのは、これらを M_i と同一視しないことである。engineer time があっても、権限・手順・依存関係が詰まっていれば rollback はできない。server capacity があっても、failover path がなければ M_{buffer} は小さい。vendor contract があっても、incident response に接続されていなければ $M_{\text{ext} \rightarrow \text{recovery}}$ は実効化しない。

この区別が、本補論の中心である。

4.5 代理構造消耗 $\hat{L}$

Software では真の L を直接測るのが難しいため、最初は代理構造消耗 $\hat{L}$ を事前固定する。

候補は次である。

proxy	定義の例	対応する構造消耗
boundary-crossing count	1 change が横断する module / service / ownership boundary 数	影響範囲の拡大
rollback-impossibility rate	失敗時に即時 rollback で きない change の比率	回復経路の喪失
hidden dependency score	明示 contract 外の呼び出 し、逆流依存、設定共有	因果追跡不能
special-case density	customer / environment / exception branch 密度	特例蓄積
untested branch rate	変更影響下で test coverage がない branch 比率	検証不能性
observability gap	failure localization に必 要な signal 欠落	原因同定困難

最初の pilot では、指標を増やしすぎない。推奨する最小構成は次である。

$$\begin{aligned} \hat{L}_{\text{pilot}} &= \\ &w_1 \cdot \text{boundary-crossing count} \\ &+ \\ &w_2 \cdot \text{rollback-impossibility rate.} \end{aligned}$$

この二つは、safe change continuity と bug localization の両方に直接関係する。前者は「どれだけ広い構造を一つの変更が巻き込むか」を表し、後者は「失敗時に戻せるか」を表す。LOC、cyclomatic complexity、churn、age といった raw baseline と比較しやすい点も利点である。

4.6 維持能力成分の対応づけ

Software / SaaS における M_i は、内部の維持能力成分と外部供給 channel を分けて操作化する。

layer	software interpretation	candidate signals
$M_{\text{buffer}}^{\text{int}}$	内在的に耐える力	redundancy, graceful degradation, queue/cache slack, rate limit, circuit breaker, spare capacity
$M_{\text{recovery}}^{\text{int}}$	内在的に戻す力	rollback path, restore test, patch path, incident runbook, monitoring, localization, SRE response
$M_{\text{reconfiguration}}^{\text{int}}$	内在的に作り変える力	feature flag, failover, modular replacement, boundary redesign, schema migration strategy, refactoring capacity
$M_{\text{ext} \rightarrow \text{buffer}}$	外部から供給される buffering	managed service redundancy, cloud provider failover, external capacity burst
$M_{\text{ext} \rightarrow \text{recovery}}$	外部から供給される repair	vendor incident response, external SRE, managed rollback support, upstream maintainer fix
$M_{\text{ext} \rightarrow \text{reconfiguration}}$	外部から供給される reconfiguration	consultant-led migration, upstream architectural change, external refactoring support

§3 の区別に従えば、外部供給は同列の第四成分ではなく、外部から他成分を供給する channel / externalization profile である。たとえば、vendor support が incident rollback を支援するなら $M_{\text{ext} \rightarrow \text{recovery}}$ 、managed service が redundancy を提供するなら $M_{\text{ext} \rightarrow \text{buffer}}$ 、external consultant が boundary redesign を支援するなら $M_{\text{ext} \rightarrow \text{reconfiguration}}$ と読む。

有効成分は、

$$\widetilde{M}_j = A_j(M_j^{\text{int}}, M_{\text{ext} \rightarrow j}),$$
$$j \in \{\text{buffer}, \text{recovery}, \text{reconfiguration}\},$$

として記録する。 A_j は少なくとも単調非減少である。最初の pilot では加法型を baseline として使ってよいが、外部支援が内部能力を置き換えるのか、内部能力と相乗するのかは domain ごとに検査すべきである。

ただし、本補論では外部供給依存をそれ自体で善とみなさない。外部供給 channel は短期維持を強める一方で、自律的な $M_{\text{buffer}}^{\text{int}}, M_{\text{recovery}}^{\text{int}}, M_{\text{reconfiguration}}^{\text{int}}$ を置き換えな
い場合がある。したがって、software mapping では「外部支援が何を供給している
か」と「base system 側にその能力が内在化しているか」を分けて記録する。

4.7 $M_{\text{reconfiguration}}$ の制限: architecture change と F 保存

Software では、 $M_{\text{reconfiguration}}$ に architecture change, modular replacement, boundary redesign を含める。しかしこれは §2.3 の制限を受ける。

すなわち、 $M_{\text{reconfiguration}}$ は target function F を保ったまま構造 Σ を再編する能力である。たとえば、API contract を保ったまま内部 module 境界を整理する、feature flag で段階的に新実装へ移行する、schema migration に backward-compatible path を持たせる、などは $M_{\text{reconfiguration}}$ に含めてよい。

一方、機能そのものを捨てる、SLA を下げる、対応しない顧客を切り捨てる、別プロダクトへ転換する、という変更は F の変更であり、本補論の $M_{\text{reconfiguration}}$ ではない。

5. 維持余力 profile の診断的読み

本補論の中心は、collapse profile の完全予測でも、修正方針の自動選択でもない。中心にあるのは、 M を有効維持余力の profile として分解し、総資源量や scalar M だけでは見えない支え方の差を記述することである。

同じ $\hat{L}$ 、同じ raw resource R 、同じ scalar M_{total} を持つ二つの software system でも、維持能力成分の構成が違えば、どの種類の余力が厚く、どの種類の余力が薄いかは異なりうる。本補論では、この差を「支え方の profile」として記録する。

5.1 Component readout examples

各成分は、実ドメインでは直接測れないことが多い。したがって、最初に必要なのは修正方針を選ぶことなく、各成分に対応する readout を事前固定することである。

component / channel	software readout examples
$M_{\text{buffer}}^{\text{int}}$	spare capacity, redundancy, queue/ cache slack, rate limit margin, graceful degradation
$M_{\text{recovery}}^{\text{int}}$	rollback path, restore drill, observability coverage, incident runbook, patch path, contract test
$M_{\text{reconfiguration}}^{\text{int}}$	feature flag coverage, failover design, boundary redesign, modular replacement, migration tooling
$M_{\text{ext} \rightarrow \text{buffer}}$	managed redundancy, cloud failover, external capacity burst
$M_{\text{ext} \rightarrow \text{recovery}}$	vendor escalation, external SRE, managed rollback support, upstream maintainer response
$M_{\text{ext} \rightarrow \text{reconfiguration}}$	consultant-led migration, upstream redesign, external refactoring support

この分類は、用語の名前ではなく、その readout がどの有効余力を表すかに
よって決まる。たとえば「vendor support」は、それが rollback を代行するなら
 $M_{\text{ext} \rightarrow \text{recovery}}$ 、容量を提供するなら $M_{\text{ext} \rightarrow \text{buffer}}$ の readout である。

5.2 M-profile reading

本補論の最小検査標的は次である。

Comparable $\hat{L}$, comparable raw R , and comparable scalar M_{total} のもとで、維持能力成分 profile は held-out outcome または診断に対して scalar baseline にない情報を持つ。

この検査標的は、「M-profile が自動的に修正方針を返す」という主張ではない。言えるのは、raw resource や scalar M だけでは見えない有効余力の偏りが、診断や予測に追加情報を持つかどうかである。

例:

- high M_{buffer} , low M_{recovery} : 障害時にしばらく耐えられるが、復旧・原因同定・再発防止は弱い可能性がある。
- sufficient M_{recovery} , low $M_{\text{reconfiguration}}$: 局所復旧はできるが、同じ種類の変更で再発しやすい可能性がある。
- high $M_{\text{ext} \rightarrow \text{recovery}}$, low internal M_{recovery} : 外部支援で短期復旧はできるが、内部に回復能力が蓄積していない可能性がある。

5.3 Software contract-coherence diagnostics との分離

Software contract-coherence diagnostics は、software domain における強い候補である。ただし、本補論の検査標的である M の維持能力成分の分解を直接検査するものではない。DeltaLint はこの診断 track の現在の実装名である。

この track が主に観測するのは、静的コード内の未整理な前提不整合、scope mismatch、guard 欠落、順序依存、設定干渉である。これは M 側というより、局所的な $\hat{L}$ または ΔL risk の観測に近い。したがって、software contract-coherence diagnostics は本補論の主 validation には含めない。

より正確には、contract-coherence benchmark が検査しうるのは、ソフトウェア崩壊そのものではなく、分散契約矛盾という早期シグナルである。ここでの software structure は、API / caller、config / runtime、documentation / implementation、default producer / consumer、lifecycle producer / consumer など、複数 surface にまたがって一貫性を保つ contract set として定義される。したがって contract-coherence benchmark は、長期保守不能化や構造死を直接測るものではなく、generic review に対して structural-lens prompt が unique valid structural root cause を増やすかを検査する operational benchmark である。

本補論では、software contract-coherence diagnostics とその現在の実装名である DeltaLint を次のように位置づける。

- Software contract-coherence diagnostics は本補論の M -framework の実証柱ではない。

- この track は LLM companion I の unscoped contradiction / attribution repair に近い L-side static-code extension として、別 note で扱う。
- DeltaLint の既存実績は、本補論においては動機づけ以上には使わない。
- DeltaLint が M_{recovery} に関与するのは、triage、patch、CI gate、rollback、migration などの repair workflow に接続された場合に限られる。

この分離により、本補論は M 側の薄い主張を保ち、software contract-coherence diagnostics は静的コードにおける L-side predictor として、独立の baseline-controlled validation を持てる。

5.4 非主張

本節では、次を主張しない。

1. Software / SaaS が仕様固定構造層であるとは主張しない。
2. $\hat{L}$ が真の L と同一であるとは主張しない。
3. M_i が単一の universal metric で測れるとは主張しない。
4. DeltaLint 実装による既存実績だけで本補論が実証されたとは主張しない。
Software contract-coherence diagnostics は本補論の主 validation から切り離し、LLM companion I / L-side の static-code extension として別 note で扱う。
5. 外部供給 channel が常に望ましいとは主張しない。外部支援は短期維持を助けるが、自律的能力を代替しない場合がある。
6. $M_{\text{reconfiguration}}$ によって F 自体を変更してよいとは主張しない。
7. 経験的検証プロトコル

本補論は、 M の完全理論を完成させるものではない。したがって本節の役割は、本補論の検査標的をどのようなデータで検証可能にするかを固定することである。

本補論の第一検査標的は、次の形を持つ。

raw resource R や既存 baseline を固定しても、M-profile readout は scalar baseline にない予測・診断情報を持つ。

6.1 検証対象

本補論では、software / SaaS を 構造推定層として扱う。第一段階の broad target と pilot target は、§4.2 で定義した通りである。

$$F_{\text{broad}} = \text{safe change continuity}$$

$$F_{\text{pilot}} = \text{change-introduced bug detection / localization}$$

ただし、M-side validation の主 outcome は単なる「bug が見つかるか」ではない。次のような operational outcome を用いる。

outcome	意味	対応する成分
change failure rate	変更が失敗・rollback・hotfix を要した比率	$M_{\text{buffer}}, M_{\text{recovery}}$
MTTR	障害から復旧までの時間	M_{recovery}
MTTD	障害検出までの時間	M_{recovery}
rollback success	rollback が即時・安全に成立したか	M_{recovery}
incident recurrence	同種障害が再発したか	$M_{\text{reconfiguration}}$
lead time degradation	変更リードタイムが悪化したか	$M_{\text{reconfiguration}}, \hat{L}$
external escalation rate	vendor / external SRE / upstream maintainer に依存した比率	$M_{\text{ext} \rightarrow j}$

この表は outcome 候補であり、実験時には対象組織・対象 repo・対象運用ログに応じて観測可能なものを事前固定する。

6.2 分析単位と分割

単位は、project / service / repository / team / time window の組で定義する。たとえば、

$$u = (\text{repo}, \text{month})$$

または

$$u = (\text{service}, \text{deployment window})$$

を一単位とする。

Primary split は time-split held-out validation とする。

$$\begin{aligned}\text{train} &: [T_0, T_1), \\ \text{test} &: [T_1, T_2)\end{aligned}$$

この分割は、software outcome が時間的に漏れやすいためである。将来の incident や bug-fix を予測するなら、未来の情報を predictor 設計に混ぜてはいけない。

Secondary split は leave-one-project-out validation とする。

$$\begin{aligned}\text{train} &: \mathcal{P} \setminus \{p\}, \\ \text{test} &: \{p\}\end{aligned}$$

これは cross-project generalization を見るためである。time-split が通っても、単一 project 内の局所慣習を拾っているだけなら、本補論の一般性は弱い。

6.3 Predictor families

比較する predictor family は、事前に固定する。

Baseline predictors

最低限、次を含める。

baseline	predictors
raw size	LOC, file count, module count
age	file age, service age
complexity	cyclomatic complexity, nesting depth, dependency count
activity	churn, commit count, author count
history	prior incidents, prior bug-fix count, prior rollback count
scalar resource	team size, on-call hours, budget proxy, server capacity
scalar M_{total}	component signals を合計または単一スカラー化したもの

これらは quality-blind / scalar-resource baseline である。本補論の検査標的は、これらの baseline より component-aware predictor が out-of-sample で勝つかにかかっている。

B2 scalar M_{total} baseline は straw-man にならないよう、train fold 上で学習した

weighted combination を用いる。test fold には固定 weight を適用する。デフォルト候補は component signal の z-score 平均である。これは本補論の component-aware model が最も強い single-scalar summary を倒すことを担保するための guardrail である。

Structure-loss proxy

第一段階の $\hat{L}_{\text{pilot}}$ は、§4.5 の最小構成を用いる。

$$\begin{aligned} \hat{L}_{\text{pilot}} &= \\ &w_1 \cdot \text{boundary-crossing count} \\ &+ \\ &w_2 \cdot \text{rollback-impossibility rate.} \end{aligned}$$

ここで、boundary-crossing count と rollback-impossibility rate の定義は outcome 観測前に固定する。重み w_1, w_2 を学習する場合は train fold のみで推定し、test fold には固定済みの重みを適用する。

Component predictors

§4.6 の component / channel 表に従い、候補 signal を事前固定する。

component/channel	candidate signals
$M_{\text{buffer}}^{\text{int}}$	redundancy, spare capacity, queue/ cache slack, rate limit, circuit breaker
$M_{\text{recovery}}^{\text{int}}$	rollback path, restore drill, observability coverage, incident runbook, patch path
$M_{\text{reconfiguration}}^{\text{int}}$	feature flag coverage, migration tooling, modular replacement path, boundary redesign capacity
$M_{\text{ext} \rightarrow \text{buffer}}$	managed redundancy, cloud failover, external capacity burst
$M_{\text{ext} \rightarrow \text{recovery}}$	vendor incident response, external SRE support, upstream maintainer response

$M_{\text{ext} \rightarrow \text{reconfiguration}}$

external migration support,
consultant-led redesign, upstream
architectural support

各 signal は、raw resource ではなく effective amount として operationalize する必要がある。たとえば「SRE がいる」は raw R であり、「対象サービスで restore drill が実施済みで手順が有効」は $M_{\text{recovery}}^{\text{int}}$ signal である。

6.4 Model families

最小比較では、次の model family を使う。

model	predictors	役割
B0 raw	raw size, age, churn, complexity	quality-blind baseline
B1 history	B0 + prior incidents / prior fixes	history baseline
B2 scalar resource	B1 + raw R / scalar M_{total}	scalar M baseline
S1 loss-aware	B1 + $\hat{L}_{\text{pilot}}$	L-side control
S2 component-aware	B1 + $\hat{L}_{\text{pilot}}$ + $\widetilde{M}_{\text{buffer}}, \widetilde{M}_{\text{recovery}}, \widetilde{M}_{\text{reconfiguration}}$	本補論の primary model

Primary comparison は、

$$S2 < B2$$

を held-out log loss / Brier score で見る。ここで $<$ は loss が小さいことを表す。

本補論の基本的な empirical claim は、S2 が B2 に対して baseline にない情報を持つかである。

6.5 Primary predictive endpoint

第一の endpoint は、held-out outcome の予測性能である。

候補は次のいずれかを事前固定する。

- binary outcome: change failure / no change failure;
- binary outcome: rollback succeeded / failed;
- binary outcome: incident recurrence / no recurrence;
- time-to-event: time to recovery, time to recurrence;

- count outcome: incidents per window.

binary outcome の場合、primary metric は Brier score または log loss とする。
count outcome の場合は Poisson / negative-binomial log loss、time-to-event の場合は事前固定した survival metric を用いる。

第一段階の safest default は binary outcome + Brier score である。理由は、calibration の解釈がしやすく、small-N の pilot でも破綻しにくいからである。

6.6 ρ_i normalization robustness

Component signal は単位が揃っていない。したがって、各 signal を $q_i = \rho_i(M_i)$ に正規化する必要がある。

ここで恣意性が入る。したがって、 ρ_i は単一に決め打ちせず、複数の合理的候補で robustness を検査する。

例:

component	normalization candidates
M_{buffer}	min-max within train fold; percentile rank; log(1+capacity proxy)
M_{recovery}	inverse MTTR percentile; rollback success rate; restore drill recency score
$M_{\text{reconfiguration}}$	feature-flag coverage; migration tooling availability; recurrence reduction proxy
$M_{\text{ext} \rightarrow \text{recovery}}$	external response SLA score; vendor escalation success; upstream fix latency inverse

All normalization parameters must be fitted on train folds only.

ここでいう「reasonable normalization」とは、本節で preregister された ρ_i 候補 family に属するものを指す。観測後に追加した normalization でのみ結果が成立する場合、それは本補論の support ではなく exploratory result として扱う。

6.7 Φ robustness

§2.5 で述べた通り、本補論は Φ の一意性を主張しない。したがって、M-profile support は Φ の選択に対して robust でなければならない。

検査する候補は次である。

family	form	interpretation
additive scalar baseline	$\sum_i \alpha_i q_i$	scalar M baseline
product	$\prod_i q_i^{\alpha_i}$	complementary components
CES	$(\sum_i \alpha_i q_i^\rho)^{1/\rho}$	substitutability
		continuum
bottleneck / Leontief	$\min_i w_i q_i$	weakest-component dominance

ここでいう「複数候補」は、preregistration で固定された Φ family に限る。観測後に追加した aggregator は探索的解析として報告できるが、primary / strong support の判定には用いない。

6.8 A_j internal/external aggregation robustness

外部供給 channel と内部能力を合わせる関数

$$\widetilde{M}_j = A_j(M_j^{\text{int}}, M_{\text{ext} \rightarrow j})$$

も一意には決まらない。ここにも robustness check が必要である。

候補:

family	form	reading
additive	$u + v$	internal and external support add
substitutive	$\max(u, v)$	stronger channel dominates
complementarity	$\sqrt{uv}$ or product-like form	both internal and external are needed
bottleneck	$\min(u, v)$	external support fails without internal uptake

特に Case C (high $M_{\text{ext} \rightarrow \text{recovery}}$, low internal M_{recovery}) では、この選択が重要になる。外部支援が短期には効くが、再発低減には内部 M_{recovery} が必要という予測は、additive だけではなく、substitutive / bottleneck 的な読みでも保たれるかを確認する。

A_j についても、reasonable candidate は preregistration で固定された family に限る。観測後に追加した A_j でのみ主張が成立する場合、それは本補論の support ではなく exploratory result として扱う。

6.9 Support criteria

本補論の validation は、段階を分けて報告する。

support level	criterion
M-profile support	component-aware predictor improves held-out risk prediction or diagnosis over raw / scalar baselines
robustness support	M-profile support is robust across ρ_i , Φ , and A_j candidate families
no-support	scalar baselines match or beat component-aware models, or results depend on post-hoc normalization / aggregation choices

この分類により、「M readout が baseline にない情報を持った」ことだけを本補論の検査標的として扱う。

ここで reasonable とは、§6.6 の ρ_i 、§6.7 の Φ 、§6.8 の A_j で preregister された候補 family に属するものを指す。

6.10 Minimum preregistration checklist

実験前に、少なくとも以下を凍結する。

- 対象 project / service / repo 集合。
- time cutoff と train/test split。
- target function F 。
- outcome definition。
- $\hat{L}_{\text{pilot}}$ の定義。
- raw baseline predictors。
- component / channel signals。
- ρ_i normalization candidates。
- Φ candidate families。
- A_j candidate families。
- primary metric。
- M-profile support threshold。

- minimum unit count per fold.
- detectable effect size / power target for the primary endpoint.
- handling of missing operational data.

これらを観測後に選ぶなら、本補論の validation ではなく post-hoc analysis である。

6.11 本節の非主張

本節は、次を主張しない。

1. 現時点で本補論の M-framework が実証済みであるとは主張しない。
2. Software / SaaS が仕様固定構造層であるとは主張しない。
3. $\hat{L}_{\text{pilot}}$ が真の L であるとは主張しない。
4. Software contract-coherence diagnostics が本補論の validation であるとは主張しない。
5. 単一の ρ_i 、 Φ 、 A_j が全ドメインで正しいとは主張しない。
6. 限界と次段階

本補論の貢献は、支える側の項 M を有効維持余力として読み直し、raw resource と同一視せず、必要に応じて維持能力成分と外部供給 channel に分け、設計・診断・比較可能な profile として扱うことである。

ただし、本補論はこの段階で empirical pilot を完了したとは主張しない。本補論が与えるのは、次の三点である。

1. $F/\Sigma/R/M$ の操作的分解。
2. software / SaaS における component-based M-profile validation.
3. それらを検査するための preregistered validation protocol.

したがって、本補論の現在地は「実証済み論文」ではなく、「強い実証可能性を持つ framework / protocol paper」である。

6.1 Operational dataset の未取得

§6 の validation は、単なる静的 code score では足りない。少なくとも次の情報が必要である。

- 対象 project / service / repository の集合。
- time cutoff と train/test split。
- target function F 。
- $\hat{L}_{\text{pilot}}$ の事前固定。
- M_{buffer} , M_{recovery} , $M_{\text{reconfiguration}}$ と外部供給 channel の component / channel signals。
- outcome: change failure rate, escaped defects, MTTR, MTTD, rollback success, incident recurrence など。

この operational dataset がない場合、risk prediction の改善は M-profile support ではなく preparatory evidence に留まる。

6.2 Software は構造推定層であり仕様固定構造層ではない

Software / SaaS は、本補論の最初の具体ドメインとして扱いやすい。しかし、SAT や Mixed-CSP のように、問題設定そのものから自然測度 m と縮小列 $V^{(0)} \supseteq V^{(1)} \supseteq \dots$ が与えられる 仕様固定構造層ではない。

安全な変更経路や有効運用状態の集合を概念的に置くことはできるが、その残存比率を domain-intrinsic に数えることは難しい。したがって、本補論の software claim は構造推定層の operational prediction であり、仕様固定構造層の普遍法則宣言ではない。

6.3 $\hat{L}_{\text{pilot}}$ は真の L ではない

§4-6 で用いる $\hat{L}_{\text{pilot}}$ は、boundary-crossing count, rollback-impossibility rate などから作る実用 proxy である。これは Paper 1 の対数比の構造消耗 L そのものではない。

この違いは重要である。

- Paper 1 の L は、構造維持可能集合の測度比から定義される。
- 本補論の $\hat{L}_{\text{pilot}}$ は、software process で観測可能な structural-risk signal から作る proxy である。

したがって、 $\hat{L}_{\text{pilot}}$ が outcome を予測しても、それだけで Paper 1 の最小形式が software に直接適用されたとは言わない。本補論で問うのは、proxy と維持能力成分の分解が out-of-sample で scalar baselines にない情報を持つかである。

6.4 Component signal は直接測定ではない

$M_{\text{buffer}}^{\text{int}}$, $M_{\text{recovery}}^{\text{int}}$, $M_{\text{reconfiguration}}^{\text{int}}$, $M_{\text{ext} \rightarrow \text{buffer}}$, $M_{\text{ext} \rightarrow \text{recovery}}$, $M_{\text{ext} \rightarrow \text{reconfiguration}}$ は、いずれも直接観測される物理量ではない。§4.6 の signal は、それぞれの成分を近似する operational indicator である。

このため、component signal の選択は preregistration で固定しなければならない。観測後に都合のよい signal を選ぶなら、それは本補論の validation ではなく、post-hoc interpretation である。

また、同じ observed signal が複数の成分に関係する場合がある。たとえば feature flag は一見 $M_{\text{reconfiguration}}$ の signal だが、rollback workflow と結びつくと M_{recovery} にも寄与する。このような signal は、どの成分の proxy として使うかを実験前に固定する必要がある。

6.5 ρ_i , Φ , A_j の一意性は主張しない

本補論は Φ の universal form を主張しない。また、各成分の normalization ρ_i や、内部能力と外部供給を合成する A_j についても、単一の正しい形を主張しない。

したがって、M-profile support についても、単一の都合のよい normalization だけで成立する場合は探索的結果に留める。

この点で、§2.5 の product / CES / bottleneck family は、表現定理の勝利宣言ではない。それらは、representation sensitivity を検査するための候補族である。

6.6 静的形式に留まる

本補論は主に静的な形式

$$S = \Phi(M)e^{-L}$$

または、その software proxy 版を扱う。時間発展としての $\dot{L}$, $\dot{M}$, collapse profile, recovery profile は本補論の主対象ではない。

これは特に $M_{\text{ext} \rightarrow \text{recovery}}$ の解釈で重要である。外部支援は短期には強い repair capacity を供給しうる。しかし、内部 $M_{\text{recovery}}^{\text{int}}$ が形成されなければ、同じ failure class の再発低減にはつながらない可能性がある。この短期/長期の差は、本補論では M-profile 上の設計仮説として述べるに留め、動的理論としては扱わない。

動的拡張では、少なくとも次を扱う必要がある。

- $\dot{L}$: 構造消耗がどの速度で増えるか。
- $\dot{M}_j$: 各成分が投資・運用変更によってどの速度で変化するか。
- 外部供給が内部能力の形成を促進するか、代替してしまうか。
- collapse / recovery の time profile。

これらは future work である。

6.7 Domain generalization は未完

本補論は software / SaaS を最初の構造推定層として扱う。しかし、 M の維持能力成分の分解は、組織、学校、病院、企業、研究チームなどにも自然に現れる可能性がある。

この cross-domain extension は本補論の主張ではない。本補論では、software-centered に保つ。

将来的には、次のような比較表を作れる可能性がある。

domain	M_{buffer}	M_{recovery}	$M_{\text{reconfiguration}}$	external supply
software / SaaS	redundancy, slack	rollback, runbook	feature flag, modular replacement	vendor / SRE support
hospital	beds, staff slack	triage, recovery protocol	protocol redesign	external transfer / regional support
school	substitute capacity	remedial support	curriculum re-configuration	district / external specialists
organization	buffer resources	incident response	structural re-configuration	consultants / external service

ただし、この表は future work であり、本補論の empirical support ではない。

7.10 Software contract-coherence diagnostics は並行 track である

Software contract-coherence diagnostics は、本補論の main validation ではない。この track が観測しているのは、主に静的コード中の未整理な前提不整合、

scope mismatch、guard 欠落、順序依存、設定干渉である。これは M の維持能力成分の構成ではなく、L-side / LLM companion I static-code extension に近い。DeltaLint はその実装名である。

したがって、この track は別 note で扱う。その中心予測は、本補論の M-profile validation ではなく、次である。

generic review + structural lens

>

generic review.

同じ model、同じ frozen context、同じ one-pass budget の下で、structural lens が generic review より unique valid structural root causes を増やすかを検査する。これは本補論の validation ではなく、LLM companion I / L-side の別 track である。将来的には、既存 static tools への追加価値や、検出された分散契約矛盾が後の bug-fix、rollback、regression、maintenance slowdown を予測するかを調べる longitudinal track へ進める。ただし、その段階までは software collapse の直接検証とは呼ばない。

7.11 次段階の研究課題

次段階の研究課題として、少なくとも次の三点が残る。

第一に、本補論用の operational dataset の選定と収集である。§6 の validation protocol は、software / SaaS 系の outcome に関する dataset を必要とする。SRE / DevOps / incident-management dataset、software delivery metrics、社内 operational log などが候補になる。どの dataset がどの程度 §6.3-6.6 の predictor および outcome に対応できるかは、今後の調査課題である。

第二に、四ドメイン比較 (software 以外への拡張) の作成である。組織、学校、病院、企業、研究チームなどにも維持能力成分の分解の自然な対応候補がある (§7.9)。ただし、これは future-work note として独立に起草するのが望ましく、本補論の empirical support には含めない。

第三に、software contract-coherence diagnostics / LLM companion I static-code extension の Phase 2 preregistration への拡張である。これは本補論の validation ではなく、LLM companion I の L-side 延長として別 track で進める。

7 結論

本補論は、構造持続の最小形式 $S = Me^{-L}$ の右辺のうち、従来から理解されてきた支える側の項 M を有効維持余力として整理した。 M の最小意味は、対象構造を維持するためにその時点で実際に使える有効量である。必要な場合には、内部の維持能力成分を $M_{\text{buffer}}^{\text{int}}, M_{\text{recovery}}^{\text{int}}, M_{\text{reconfiguration}}^{\text{int}}$ に分け、外部供給 channel を $M_{\text{ext} \rightarrow \text{buffer}}, M_{\text{ext} \rightarrow \text{recovery}}, M_{\text{ext} \rightarrow \text{reconfiguration}}$ として、それぞれの実効能力を $\widetilde{M}_j = A_j(M_j^{\text{int}}, M_{\text{ext} \rightarrow j})$ に集約できる。そのうえで、 Φ による effective maintenance amount $M_{\text{eff}} = \Phi(\widetilde{M}_{\text{buffer}}, \widetilde{M}_{\text{recovery}}, \widetilde{M}_{\text{reconfiguration}})$ を通じて、構造持続量を書き直せる。

本補論の固有の検査標的は、raw resource R が対象構造の維持に使える有効量 M になっているか、その操作的 readout が scalar baseline にはない情報を持つかを調べることである。成分分解はそのための補助的な profile である。本補論はこの検査標的を software / SaaS を最初の構造推定層として具体化し、 ρ_i, Φ, A_j の候補族に対する頑健性検査を含む、事前固定可能な経験的検証プロトコルを定式化した。実際の preregistration と pilot 実行は、本補論の外、別の empirical program として進める。

本補論は新しい普遍法則の証明ではなく、また empirical pilot 完了論文でもない。本補論の位置づけは、構造持続の収支原理の回復量・資源入力を実ドメインで測るための support-side operational mapping である。構造持続の最小形式と条件つき導出補論が構造消耗側の対数比の構造消耗を特徴づけ、LLM companion I と II が構造消耗と支援の相互作用を経験的に観察したのに対し、本補論は support 側の操作的座標系を提供する。そこから自然に出てくる次段階は、準備された protocol を operational data に適用する経験的 pilot であり、それは本補論の外、別の empirical program として進める。